# LABORATOR REȚELE

## Capitolul 1 - Programming Basics

**Docker, Modelul OSI + TCP/IP, Adrese IP, netcat, traceroute, Manipularea Datelor (Endianness), Python**

**Martie 2026**

# Recap: Adrese IP și Subnetting

- adresă IPv4 - stocată în calculator ca 32 de biți (noi o scriem în format zecimal punctat pentru lizibilitate)

```
Adresa:  192    .  168   .   5    .   1
Binar:   11000000.10101000.00000101.00000001
```

- **Cum convertim manual un octet?**
  - Fiecare octet are 8 biți, cu valorile poziționale:

```
Poziție:  128  64  32  16   8   4   2   1
    Bit:    1   1   0   0   0   0   0   0  = 128 + 64 = 192

192 = 128 + 64          → 11000000
168 = 128 + 32 + 8      → 10101000
5   = 4 + 1             → 00000101
1   = 1                 → 00000001
```

# Recap: Adrese IP și Subnetting

- Orice adresă IP dintr-o rețea are două componente:
  - **Prefixul de rețea** (fix) = identifică rețeaua
  - **Sufixul de host** (variabil) = identifică dispozitivul în rețea

- **Masca de rețea (netmask)** specifică câți biți din stânga sunt prefixul de rețea (/x)

Exemplu pt. 192.168.5.0/24 :

```
Adresa IP:    192.168.5.1   = 11000000.10101000.00000101.00000001
Masca /24:    255.255.255.0 = 11111111.11111111.11111111.00000000
                              <-----rețea (24 biți)------><-host->

     Adresa rețelei:  192.168.5.0   (toți biții de host = 0)
     Prima adresă:    192.168.5.1   (primul host utilizabil)
    Ultima adresă:   192.168.5.254 (ultimul host utilizabil)
     Broadcast:       192.168.5.255 (toți biții de host = 1)
         Hosturi utile:   254           (2^8 - 2 = 254)
```

Hosturi utile = 2^(32 - prefix) - 2

-2 deoarece:

- adresa rețelei (x.x.x.0) nu poate fi asignată
- adresa broadcast (x.x.x.255) nu poate fi asignată

# Recap: Adrese IP și Subnetting

## Exerciții!

- Care este intervalul de IP-uri al adresei de retea 64.183.234.9/15?
- Cate IP-uri pot fi atribuite in reteaua 192.168.0.0/16?
- Care este ultimul IP din reteaua 164.88.24.37/29?

# Recap: Adrese IP și Subnetting

## Soluții!

- Care este intervalul de IP-uri al adresei de retea 64.183.234.9/15?

/15 înseamnă că primii 15 biți sunt fixați.
64.183.234.9 în binar:
  64 = 01000000  183 = 10110111  234 = 11101010  9 = 00001001

/15 → primii 15 biți sunt rețeaua:
  01000000.1011011[1] → bitul 16 (ultimul din octetul 2) face parte din host

Masca /15 = 11111111.11111110.00000000.00000000 = 255.254.0.0

Adresa rețelei: 64.182.0.0  (resetăm biții de host la 0)
  Octet 2: 183 = 10110111, masca octet 2: 11111110
  183 AND 254 = 10110110 = 182

Intervalul: 64.182.0.0 → 64.183.255.255

# Recap: Adrese IP și Subnetting

## Soluții!

- Cate IP-uri pot fi atribuite in reteaua 192.168.0.0/16? = 2^(32-16) - 2 = 2^16 - 2 = 65536 - 2 = 65534

- Care este ultimul IP din reteaua 164.88.24.37/29?

/29 → primii 29 biți = rețea, ultimii 3 biți = host (max 32 biți)
Masca: 255.255.255.248 = 11111111.11111111.11111111.11111000

Octetul 4: 37 = 00100101
Masca oct4: 11111000 → pe octetul 4: 0 0 1 0 0 | 1 0 1
Rețea: 0 0 1 0 0 | 0 0 0 (= 32)
Broadcast: 0 0 1 0 0 | 1 1 1 (= 32 + 4 + 2 + 1 = 39)

Adresa rețelei: 164.88.24.32
Broadcast: 164.88.24.32 + 7 = 164.88.24.39 (2^3 = 8 adrese, ultimul = 39)

Ultimul IP utilizabil: 164.88.24.38
Ultimul efectiv = broadcast = 164.88.24.39

# netcat: Comunicare Client/Server

- **socket** = capăt al unei conexiuni de rețea.
  - ca un telefon: un receptor care așteaptă apeluri (server) și un apelant (client)
  - indentificat prin:
    - Adresa IP (pe ce mașină se află)
    - Portul (pe ce "canal" ascultă - număr 0–65535)
    - Protocolul (TCP/UDP)

- **netcat** = un utilitar de rețea extrem de simplu care poate:
  - deschide un socket și asculta pe un port (mod server)
  - conecta la un socket existent și trimite date (mod client)

**LIVE DEMO Docker:**

**Terminal 1 - Serverul (în rt1):**

```
>> docker compose exec rt1 bash
>> nc -l 8081
```

→ se deschide un socket TCP pe portul 8081 și nu face nimic până când un client se conectează

**Terminal 2 - Clientul (în rt2):**

```
>> docker compose exec rt2 bash
>> echo "salut din rt2" | nc <adresă IP rt1>
8081
```

→ **Ce se întâmplă?**

# traceroute

- = utilitar care arată fiecare router (hop) pe care îl traversează un pachet de la sursă la destinație, împreună cu latența la fiecare hop.

- **Cum funcționează**: traceroute explotează TTL (Time To Live)
  - TTL = câmp din headerul IP care se decrementează cu 1 la fiecare router traversat.
  - Când TTL ajunge la 0, routerul aruncă pachetul și trimite înapoi un mesaj ICMP ”Time Exceeded”
  - **Pas 1**: Trimite pachet cu TTL = 1 → Primul router decrementează la 0, aruncă pachetul → Trimite ICMP "Time Exceeded" înapoi → traceroute înregistrează: Hop 1 = IP router, latență = X ms
  - **Pas 2**: Trimite pachet cu TTL = 2 → Primul router decrementează la 1, pasează mai departe → Al doilea router decrementează la 0, aruncă → Trimite ICMP "Time Exceeded" → traceroute înregistrează: Hop 2 = IP router, latență = Y ms

  ... și tot așa până când pachetul ajunge la destinație

# traceroute

!! posibil să nu meargă pe Docker Desktop pt Windows/mac, known bug → încercați local în cmd/terminal tracert pt Windows, traceroute pt mac

**LIVE DEMO Docker:**

```
>> docker compose exec rt1 bash
>> traceroute www.google.ro
```

**Output exemplu:**

```
>> traceroute to www.google.ro (142.250.185.163), 30 hops max
  1     2 ms     1 ms     8 ms  192.168.1.1 [192.168.1.1]
  2     3 ms     4 ms     3 ms  10.0.33.252 [10.0.33.252]
  3     3 ms     2 ms     2 ms  10.12.100.33 [10.12.100.33]
...
..8    16 ms    15 ms    15 ms  72.14.216.212
  9    17 ms    17 ms    17 ms  172.253.51.195
 10    15 ms    15 ms    14 ms  209.85.244.147
 11    15 ms    16 ms    16 ms  fjr04s12-in-f14.1e100.net [172.217.17.78]
```

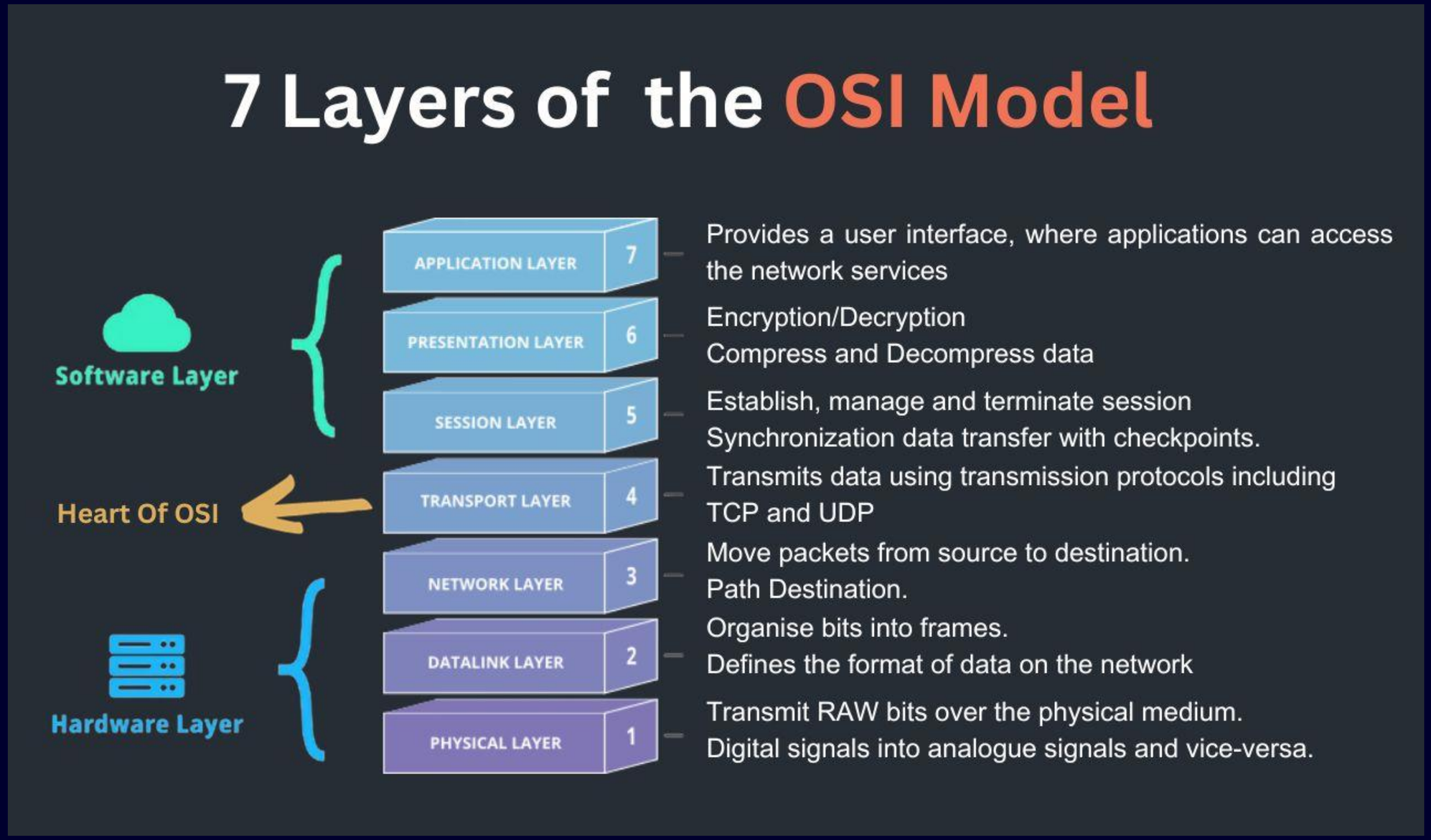
1. DNS a găsit IP pt google → 142.250.185.163
2. Prima rețea - IP privat, LAN, timp răspuns mic, echipament fizic apropiat
3. 10.x.x.x - tot rețele private! Fie rețea f mare, sau rețeaua privată a ISP

- Carrier-Grade NAT (CGNAT) → Pachetul trece prin multiple routere interne ale providerului din România pentru a fi agregat cu traficul altor utilizatori, înainte de a ieși pe o singură adresă IP publică a ISP-ului

1. 72.x.x.x → am intrat pe internet! rețeaua Google
2. 1e100.net = domeniul intern Google (10 la puterea 100 = un Googol)

Multe * = routerul a primit pachetul, dar nu a răspuns cu ICMP Time Exceeded - e configurat să nu răspundă (firewall)/răspunsul s-a pierdut

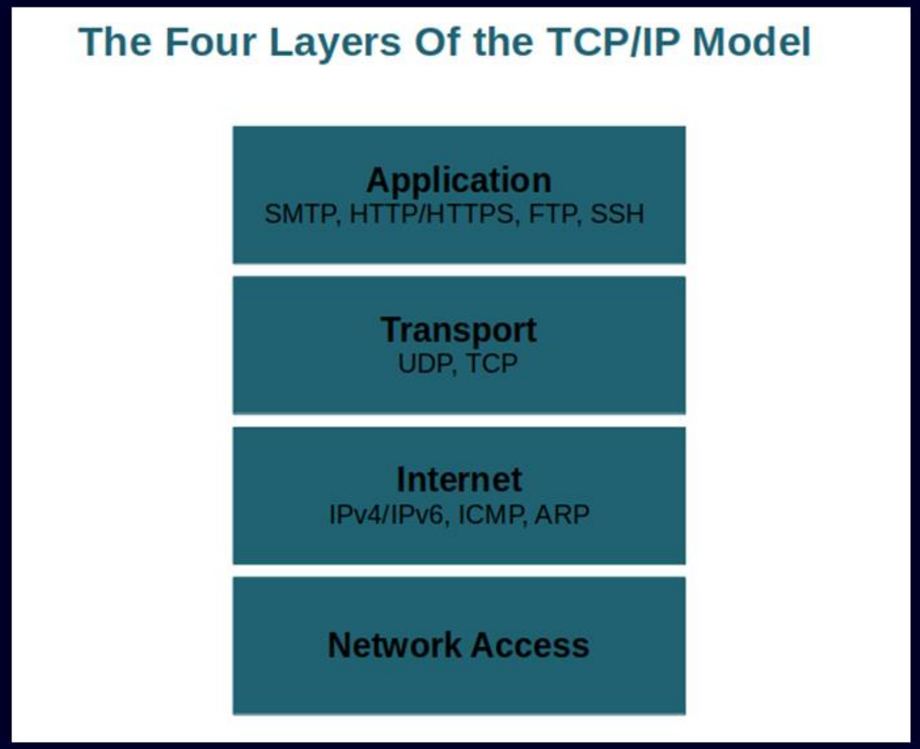
# Recap OSI Model




**! Model teoretic, nu implementare efectivă**

Fun fact: A fost creat de US Department of Defence

# TCP/IP Model: Implementarea reală




- **Application Layer**: Interfața cu utilizatorul și programele (HTTP - accesare web, FTP - file transfer, SMTP - trimitere email, DNS - traducere nume domenii)
- **Transport Layer**: Gestionat de TCP/UDP, fragmentează datele în segmente, asigurând livrarea corectă și ordonată a acestora, reasamblându-le la destinație
- **Internet Layer**: Folosește IP (Internet Protocol) pt. a încapsula segmentele în pachete și a le ruta prin cea mai bună cale către destinație
- **Network Access Layer**: Gestionează transmisia fizică a datelor (cabluri, Wi-Fi), adresarea fizică și accesul la mediul de rețea

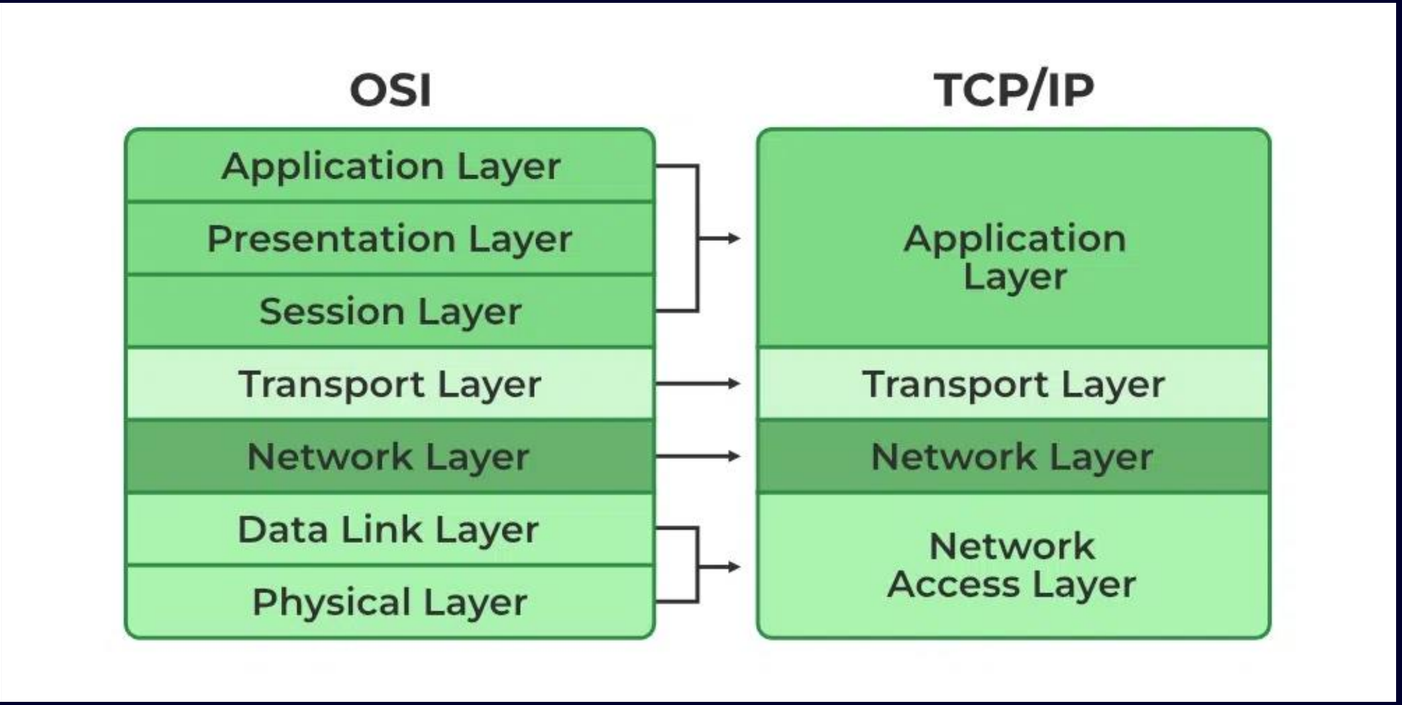
! Fiecare strat ia datele de la stratul superior, adaugă propriul header (antet) cu informații specifice și le dă mai departe (= încapsulare)

| | |
|---|---|
| Date aplicație: | [Mesaj HTTP] |
| Transport: | [TCP Header][Mesaj HTTP] |
| Internet: | [IP Header][TCP Header][Mesaj HTTP] |
| Network Access: | [MAC Header][IP Header][TCP Header][Mesaj HTTP][MAC Trailer] |

# De ce nu OSI?

- Modelul OSI a fost formalizat după ce rețelistica începuse deja să adopte protocoale practice, TCP/IP era deja funcțional și folosit la scară largă

- OSI, cu 7 straturi, este mult mai complex și ”teoretic”, pe când TCP/IP e mai flexibil, simplu, mai practic ca fundație pentru internet




**De ce e totuși important OSI?** Este precum ”gramatica” unei limbi vorbite (aici, TCP/IP) - formalizează vocabularul și logica din spatele modelului practic

# Problema antetelor: Endianness

Endianess = ordinea în care octeții (bytes) sunt așezați în reprezentarea datelor

Articol care vorbește mai mult despre Endian(m)ess :)

- Am stabilit că TCP/IP adaugă "Headers" (ex: Portul destinație 80). Dar pe cablu, acel port este doar un șir de semnale electrice (biți/octeți).

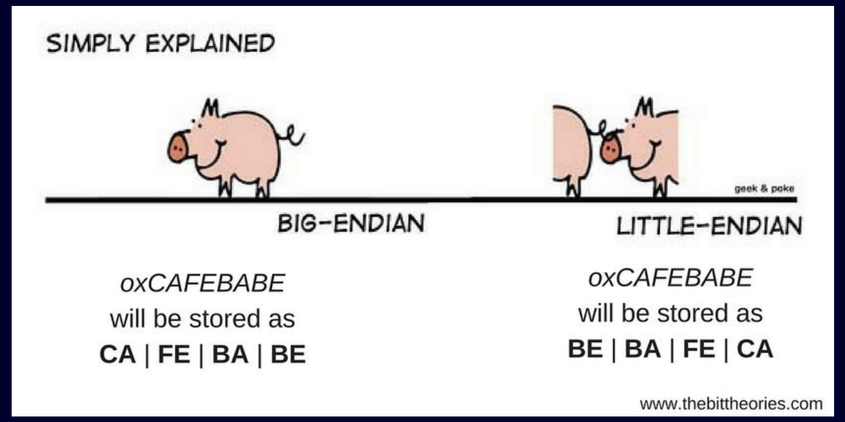
- **Analogia Datei:** Dacă scriu data 05/04/2026, ce înseamnă?
  - RO: 5 Aprilie;
  - SUA: 4 Mai

  → Datele nu s-au pierdut pe drum, dar ordinea de citire a fost greșită, ducând la un mesaj corupt.

**Exact aceeași problemă la Host (device-ul nostru) order vs Network order**

# Network vs Host Order

- Network Order = Big Endian (trimite octeții de la stânga la dreapta, primul octet = MSB = most singificant byte)
- Host Order = Little Endian (trimite octeții de la dreapta la stânga, primul octet = LSB = least significant byte)




Ex: numărul hexazecimal 0x12345678:

- LE (Host): 78 56 34 12
- BE (Network): 12 34 56 78

**De ce sunt diferite?** → design pt. scopuri diferite; host order prioritizează eficiența CPU (calcule mai rapide) vs network order (BE) reprezintă ordinea standard de transmitere pt. majoritatea arhitecturilor hardware

# Intro Python pt. Rețele

- **De ce Python pentru Rețele?**
  - Sintaxă clară și concisă
  - Librării bune: socket, struct, scapy
  - Portabilitate: rulează identic pe orice OS

- **Exerciții în Python!**
  1. **conversia unui octet IP**
  2. **construiește adresa de rețea din IP și mască**

# Python struct: Bytes ca C Types

- Protocoalele de rețea definesc headerele în termeni de tipuri C cu lungimi fixe:
  - primii 4 biți = versiunea IP (unsigned nibble)
  - bytes 0-1 = portul sursă (unsigned short, 16 biți)
  - bytes 2-5 = numărul de secvență (unsigned int, 32 biți)

- Python lucrează cu integers de dimensiune arbitrară, nu cu tipuri C fixe
  → modulul **struct** convertește tipuri Python ↔ bytes cu tipuri C cu lungimi exacte

- **Exerciții în Python!**
  1. **Struct**
  2. **Endian**

**! Tabel complet cu C Types în repo**

# Scapy: Capturarea Pachetelor în Cod

- librărie Python care permite:
  - Capturarea pachetelor (similar cu tcpdump, dar în cod)
  - Construirea pachetelor de la zero, câmp cu câmp
  - Trimiterea pachetelor în rețea
  - Parsarea pachetelor capturate sau citite din fișiere .pcap
  - permite procesarea programatică = poți scrie logică complexă de filtrare, statistici, modificare de pachete.

Instalare:

```
docker compose exec --user root rt1
                bash
pip3 install --pre scapy[complete]
```

```
from scapy.all import *

pachete = sniff()
#Ctrl+C pentru oprire

#facem chestii pe retea, ping de pe rt2 etc.
print(pachete)

#afisarea pachet capturat
pachete[0].show()
```